

LAB 211 Assignment

Type: Long Assignment
Code: J1.L.P0039
LOC: 500
Slot(s): N/A

Title

Student and Short Course Management System

Background

A university maintains information about students and the courses they enroll in. To support academic management, the university requires a Java console-based application to manage students and the courses they enroll in.

The system stores data in two text files: **Students.txt** and **Courses.txt**.

File: Students.txt	Description
STU0001, Nguyen Van An, Software Engineering, 3.4 STU0002, Tran Thi Binh, Information Systems, 3.8 STU0003, Nguyen Tuan An, Software Engineering, 3.4 STU0004, Tran Thi Bich, Information Systems, 3.7 STU0005, Le Hoang Minh, Computer Science, 3.2 STU0006, Pham Quoc Huy, Artificial Intelligence, 3.9 STU0007, Vo Thanh Tam, Data Science, 3.5	Information on a line: <Student ID>, <Full Name>, <Major>, <GPA> Note: GPA is a double.

File: Courses.txt	Description
CSE201, STU0002, Database Systems, 10, 15/09/2025 MAT301, STU0003, Discrete Mathematics, 8, 20/09/2025 AI401, STU0004, Machine Learning, 10, 05/10/2025 DS501, STU0005, Big Data Analytics, 8, 10/10/2025 PRJ301, STU0001, Java Web Project, 6, 20/10/2025 PRO192, STU0003, Java Programming, 8, 25/10/2025 DBI202, STU0002, Database Design, 5, 01/11/2025	Information on a line: <Course ID>, <Student ID>, <Course Name>, <Duration_Weeks>, <Start_Date> Note: Student ID must exist in the students list.

Constraints

1. Constraints on Students:

- Student ID must be unique and follow the format STU0000.
- Full name cannot be empty and must contain at least two words.
- Major cannot be empty.
- GPA must be double value between 0.0 and 4.0.

2. Constraints on Courses:

- Course ID must be unique.
- Student ID must exist in the student list.

- Course name cannot be empty.
- Duration must be a positive integer in weeks, minimum **1 week**.
- Start date must be a future date.

Program Specifications

Students are required to build a Java console application using **the Object-Oriented Programming (OOP)** approach to manage students and courses in a university.

Menu Functions

- 1-List all students
- 2- Add a new student
- 3- Search for a student by ID
- 4- Update a student by ID
- 5- List all students by major
- 6- Add a new course
- 7- List all courses by student (grouped)
- 8- Calculate total study duration by student ID
- 9- Remove a student by ID
- 10- Sort students by GPA
- 11- Save data to files
- 12- Quit program

Features and Assessment

No.	LOCS	Function Description
1	20	List of all Students: Display data in a formatted table with column headers (ID, Name, Major, GPA).
2	50	Add a new Student: Accept ID, Name, Major, and GPA. Add to the list only if all constraints are satisfied. Display success/failure message.
3	20	Search for a Student by ID: If the Student does not exist, show: " Student ID does not exist! ". Otherwise, display Student information.
4	50	Update a Student's GPA by ID: If the Student does not exist, show: " Student ID does not exist! ". Otherwise, allow updating GPA (must be [0.0 to 4.0]).
5	50	List all Students by Major: Accept a Major input (e.g., "Software engineering"); display all Students whose major matches the input.
6	50	Add a new Course: Accept Course ID, Student ID, Course Name, Duration weeks and Start Date. Student ID must be selected from a menu of existing Students. Validate all constraints.

No.	LOCS	Function Description
7	50	List all Courses by Student (Grouped): Display the list of Students. Under each Student, list all Courses they are currently or have been assigned to, showing Course ID, Duration, and Start Date.
8	50	Calculate Total duration by Student ID: Accept Student ID; calculate and display the total duration the Student has spent on all registered Courses.
9	30	Remove a Student by ID: Accept Student ID. If the Student does not exist, show: " Student ID does not exist! ". If the Student has associated Courses, show: " Cannot delete: Student is assigned to Courses. ". Otherwise, remove the record.
10	30	Sort Students by GPA: Display the list of Students sorted by GPA in ascending order.
11	50	Save data to files: Save both Student and Course lists into Students.txt and Courses.txt files, respectively. Show a confirmation message.
12	50	Quit program: Check for unsaved changes. If changes exist, prompt the user: " Do you want to save the changes before exiting? (Y/N) ". Exit the program after handling the save/discard choice and reload data on program start.

Note for Students

- **OOP Principles:** Follow OOP principles strictly (encapsulation, inheritance, polymorphism, abstraction). Design appropriate classes (Student, Course, etc.).
- **Computational Thinking:** Apply Decomposition, Pattern Recognition, Abstraction, and Algorithm in your design.
- **Input Validation:** Input validation must strictly follow all defined constraints. Show appropriate, helpful messages for invalid or missing data.
- **Data Persistence:** The program must **load** data from the files upon startup and **save** data upon existing (if changes occurred) or when function 11 is selected.

Submission

- Source code
- Class Diagram
- Flow-chart